

LIST INTERVIEW QUESTIONS

- 1 Why is a Python list mutable, and what problems can mutability create?
- 2 How does Python handle object references when multiple variables point to the same list?
- 3 What is the difference between list identity and list equality?
- 4 What is the difference between assignment, shallow copy, and deep copy?
- 5 Why does shallow copying become problematic with nested lists?
- 6 How does `deepcopy()` differ from a normal list copy?
- 7 Can a list contain mutable and immutable objects together? How does this affect the list?
- 8 Why can a list contain a dictionary or another list as an element?
- 9 Why can't a list itself be used as a dictionary key?
- 10 Why can a tuple sometimes be used as a dictionary key while a list cannot?
- 11 What determines whether an object can be used as a dictionary key?
- 12 What is the difference between a nested list and multiple references to the same inner list?
- 13 Why can modifying one inner list unexpectedly modify multiple parts of a nested structure?
- 14 How does Python's reference model affect copying nested lists?
- 15 What is the difference between mutability of a list and mutability of the objects stored inside it?

16	Why does Python allow different data types to exist in the same list?
17	What happens conceptually when a list is passed as an argument to a function?
18	Why can a function modify a list passed to it without returning the list?
19	What is the difference between modifying a list and reassigning the list inside a function?
20	Why is modifying a list while iterating over it considered unsafe or error-prone?
21	How does list comprehension differ conceptually from a normal for loop?
22	What are the limitations of list comprehension?
23	When would using a list comprehension reduce code readability?
24	What is the difference between <code>append()</code> and <code>extend()</code> when dealing with iterable objects?
25	Why does <code>extend()</code> behave differently depending on the type of object passed to it?
26	What is the difference between <code>sort()</code> and <code>sorted()</code> from the perspective of mutability and returned value?
27	What is stable sorting, and why is it useful when sorting complex objects?
28	How does Python determine the order when sorting a list of custom objects?
29	In what situations would you choose a list over a tuple, set, or dictionary despite their different characteristics?
30	If a Python list contains thousands of objects of different types, what conceptual challenges can arise when processing or maintaining that list?